CSUN | COLLEGE OF ENGINEERING AND COMPUTER SCIENCE

ECE 520/L

System-on-Chip Design with Lab

Final Project
Matrix Multiplication Accelerator

Daniel Celnik
Instructor: Aaron Nanas
Spring 2026

Introduction

This project covers the design, implementation, and verification of a 1D BRAM-based matrix multiplication accelerator with software-parameterized matrix dimensions. The accelerator uses a 4-way parallel hardware multiply-accumulate (MAC) unit with an FSM-based controller to compute C = A x B, address generation hardware to map the matrix dimensions to BRAM addresses, and is controlled via AXI4-Lite control registers accessible by software via a submodule. The hardware was designed using Vivado block design tools and features Block Memory Generators (BMGs) operating in True Dual Port RAM mode; Port A is used to interface with the PS, while Port B is customized to be used with the matrix accelerator IP.

The corresponding software program loads default input square matrices with dimensions that are multiples of 4, starts the accelerator, reads the result when prompted via a hardware interrupt, and verifies the output against a software reference model. Finally, the program compares the amount of time taken to calculate the software reference with that taken by the hardware accelerator to yield a tangible representation of the speed improvement.

Components Used

- Zybo Z7 ARM/FPGA SoC Board
- USB-A to USB Micro-B Cable

Project Block Diagram

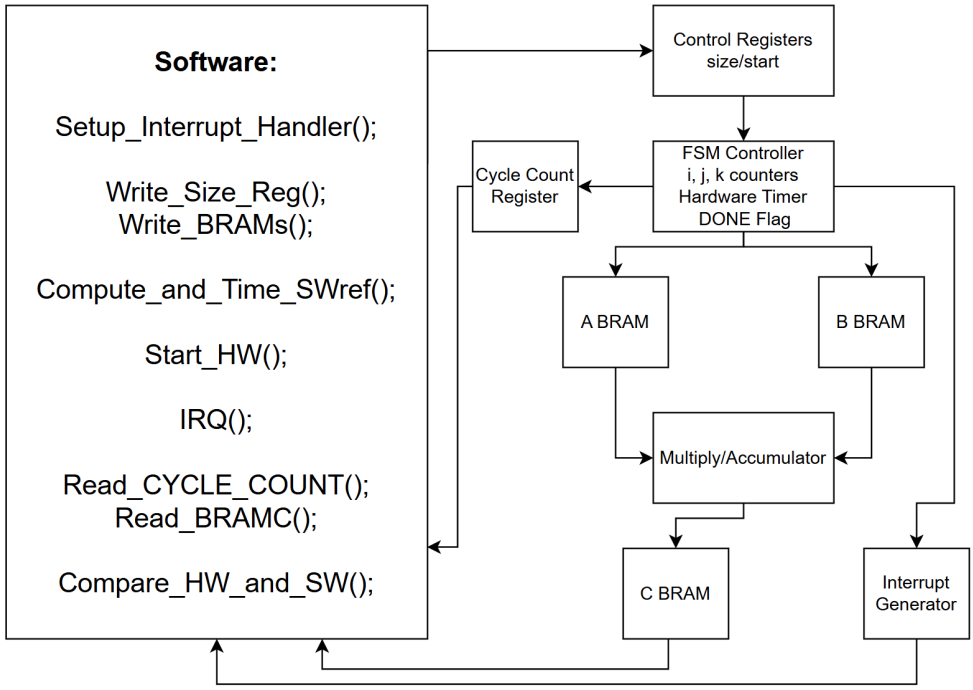

Fig. 1. Project Block Diagram

Design and Implementation

The matrix accelerator employs the basic linear algebra concept of matrix multiplication, where for input matrices A and B and output matrix C, each cell of matrix C is calculated using the formula shown below in Fig. 2.

$$C[i][j] = \sum_{k=0}^{N-1} (A[i][k] * B[k][j])$$

Fig. 2. Matrix multiplication formula

The hardware for this project was primarily designed within Vivado's block design environment to look as closely to Fig. 1 as possible. A ZYNQ7 PS block must be instantiated to supply a global clock and reset signal to the PL design. There must be 3 BMGs instantiated, one for each matrix, as well as a corresponding AXI BRAM Controller for each BMG to allow the PS to read and write from each BRAM unit. Furthermore, an AXI Interconnect block allows the PS to communicate with each AXI BRAM Controller as well as the main matrix accelerator IP. The matrix accelerator IP itself must be custom designed to have the following functionality:

1. Read from BRAM units A and B
2. Write to BRAM unit C
3. Enable the PS to read and write to AXI4-Slave control registers
4. Output an interrupt to the PS to signal that the calculation is complete
5. Output debug signals to an ILA to enable rapid troubleshooting/prototyping

Note that the first two requirements above must be independent of the PS reading from BRAM C and writing to BRAMs A and B; therefore the Dual Port RAM memory type is required for each BMG. For each BMG, Port A is reserved for the PS via AXI BRAM Controllers while Port B is reserved for the MAC IP via exposed BRAM interface signals.

The design of the matrix accelerator IP began with creating and packaging a new IP, similar to the lab procedure, to produce an AXI4 IP template complete with register read and write logic. This IP uses 4 AXI4-Slave control registers to facilitate configuration and provide I/O to the PS:

1. FSM Control
    a. Start
    b. Irq_enable
    c. Soft_reset
2. Matrix Size
3. Clock Cycle Count

4. Status

The registers corresponding to the FSM Control and Matrix Size inputs to the IP are configured as read-only, since only the PS writes to these registers. Meanwhile, the Clock Cycle Count and Status registers are configured to be writable by the IP and readable to the PS. The module that configures this slave register functionality is then instantiated in a top module that handles the BRAM address generation, matrix calculation, state selection logic, and sequential logic determining the IP's core functionality.

BRAM Address Generation

The BRAM address generation portion of this IP is responsible for feeding the correct matrix elements to the MAC at the correct time. Its initial implementation began as a simple nested-loop, similarly to a software implementation, for rapid prototyping purposes. In Fig. 2, the "k" element represents the inner dimension of matrix A's width and matrix B's length, which are always equal to each other. For each element in matrix C at a particular row "i" and column "j", the MAC must multiply corresponding elements from A and B, specifically A[i][k] and B[k][j], and accumulate the result over all values of k before resetting its value and incrementing the column index j. The process continues across all columns before advancing to the next row i. This overall operation exhibits a time complexity of O(N^3).

BRAM constructs function similarly to 1-dimensional arrays. For an NxN matrix, every N+1 element is the start of a new row. Therefore, the BRAM address corresponding to any matrix element can be calculated using the following formulas:

1. Address_A_Index = i*N + k
2. Address_B_Index = j*N + k
3. Address_C_Index = i*N + j

The majority of the time spent on this design is awarded to the process of discovering that Xilinx BRAM uses byte addressing instead of bit addressing; since there is 1 byte per 8 bits, or 4 bytes per 32 bits, the calculated address indices must be multiplied by 4 for a default-sized 32-bit data word. This is accomplished via a left shift of 2 for all calculated address indices.

Matrix Calculation

The matrix calculation is a subset of the core sequential logic during the ST_MAC state. Once the proper BRAM addresses are calculated, the corresponding matrix element values at those addresses are latched. On the next clock cycle, the accumulator is incremented by the product of A_reg_value and B_reg_value. Several cycles later, the next 2 elements are multiplied together and added to the accumulator, and so on, until all N element pairs have

had their products accumulated. The IP then writes that final accumulator value to the next available address in BRAM C. This process continues until all elements in matrix C have been filled.

State Selection Logic

The IP is controlled by a 12-state FSM that handles the determination of operation phases. All states immediately progress to the next state unless otherwise notated. The states are listed and the next-states are determined (if applicable) as follows:

1. ST_IDLE (default) - waits for start_pulse && (N != 0)
2. ST_CLEAR_ACC
3. ST_READ_AB
4. ST_WAIT_AB1
5. ST_WAIT_AB2
6. ST_LATCH_AB
7. ST_MAC
8. ST_CHECK_K - returns to ST_READ_AB if (k < N-1)
9. ST_WRITE_C
10. ST_CHECK_J - returns to ST_CLEAR_ACC if (j < N-1)
11. ST_CHECK_I - returns to ST_CLEAR_ACC if (i < N-1)
12. ST_DONE

Core Sequential Logic

Each state listed above features key functionality to ensure proper operation of the MAC IP. That functionality is described in-detail below in Fig. 3.

ST_IDLE	Resets all indices, the accumulator, and the busy flag. Once the rising edge of the start signal is detected, the FSM sets the busy flag and resets the done flag, the interrupt request, and the cycle_count register.
ST_CLEAR_ACC	Resets the accumulator and the index k.
ST_READ_AB	Enables BRAM units A and B and supplies them with the calculated addresses.
ST_WAIT_AB1	BRAM enable remains high to account for BRAM latency.
ST_WAIT_AB2	BRAM enable remains high for an extra clock cycle to ensure all data is stable.
ST_LATCH_AB	The MAC IP latches onto data from BRAM units A and B.
ST_MAC	Using the data latched in the previous state, the MAC IP accumulates their dot product.
ST_CHECK_K	The MAC IP checks the index value k relative to the total matrix width N; if (k < N-1), k is incremented.
ST_WRITE_C	The MAC IP writes the final accumulated value to BRAM unit C at matrix index [i][j].
ST_CHECK_J	The MAC IP checks the index value j relative to the total matrix width N; if (j < N-1), j is incremented and k is reset.
ST_CHECK_I	The MAC IP checks the index value i relative to the total matrix width N; if (i < N-1), i is incremented while j and k are reset.
ST_DONE	Deassert the busy flag, assert the done flag and the interrupt request.

Fig. 3. Core sequential logic of the 12-state FSM controlling the custom MAC IP.

Software Design and Validation

The main loop features a user menu interface to configure control registers and explore the functionality of the hardware. The following options are presented to the user:

1. Load default matrices into software buffers.
2. Write matrices A and B into BRAM.
3. Compute the software reference (timed).
4. Run the hardware accelerator using interrupts.
5. Read matrix C from BRAM and compare HW/SW results.
6. Print accelerator registers.
7. Print matrices A, B, C_hw, and C_sw.
8. Enable interrupts.
9. Disable interrupts.
10. Run the full matrix test using interrupts (1-5).
11. Assert the hardware accelerator's soft reset.
12. Change the matrices' sizes.
13. Print hardware control registers' base addresses and interrupt information.
14. Display this menu.

User input is determined using the built-in inbyte() function and a switch-case statement. The following describes how each menu option is implemented in software:

First, the LoadDefaultMatricies function uses a double nested for-loop to write increasing values to each matrix element in Matrix A, while Matrix B receives decreasing values to each element. It saves each element to 32-bit wide, 256-bit deep static variables called g_matrix_a and g_matrix_b. These variables must be 256-bits deep to account for the possibility of performing matrix operations on 16x16 matrices.

To write matrices A and B into BRAM, the software program uses the same algorithm discussed in the BRAM Address Generation section to calculate the address to write the contents of g_matrix_a and g_matrix_b. Since each row of Matrix A is multiplied by each column of Matrix B, the program stores a transposed version of the calculated Matrix B to be stored in BRAM B.

To compute the software reference, the XTime_GetTime function is used to track the amount of time between the start and end of the matrix multiplication calculations. A triple nested for-loop algorithm, similar to the hardware implementation, is used to calculate the sum corresponding to each value written into Matrix C_SW. After all k multiplications/additions are performed sequentially, software stores the final sum in Matrix C_SW before incrementing the outer loop indices.

To run the hardware accelerator using interrupts, software writes to the Start and Size AXI slave control registers while setting an interrupt_received signal to 0. An ISR handles the interrupt once generated by hardware, setting interrupt_received to 1 and prompting software to read the values in the Status and Cycle_Count AXI slave registers. Finally, the function calculates the time in microseconds taken by the hardware calculations by multiplying the number of hardware cycles taken by 1000000 (the number of microseconds per second) and dividing that product by the PL's clock frequency.

Reading the hardware's computed value for Matrix C requires software to read from BRAM C word-by-word, where each word corresponds to one matrix element. Since BRAMs store data in a 1-dimensional array, this is implemented via incrementing a word index variable in software. Software then compares each element of Matrix C_SW and Matrix C_HW using the same index variable to iterate through each of the g_matrix_c_sw and g_matrix_c_hw 2-D arrays, returning the number of errors.

When the user wants to print the contents of the accelerator's registers, software simply reads from each of the 4 AXI slave control registers and prints them to the terminal.

To print each of the 4 matrices (A, B, C_SW, C_HW), software references a pointer to each matrix and uses a double for-loop to iterate through each of that matrix's elements.

Enabling and disabling interrupts is implemented via reading from the Control AXI slave register, ORing that value with the IRQ bit mask when enabling, or ANDing that value with the complement of the IRQ bit mask, and writing that value back into the control register.

Asserting a soft reset to the hardware is accomplished by writing a bit mask targeting the reset bit of the Control AXI slave register before writing 0x00000000 to that same register in the next CPU instruction cycle.

Most hardware accelerator helper functions have a parameter referring to the size of each matrix's dimensions, stored as an unsigned integer n, which is modified via menu option 12.

Finally, the user can choose to print the base addresses of each hardware component, including the matrix accelerator and each of the 3 BRAM units as well as the matrix accelerator's interrupt ID and the constant corresponding to the PL's clock frequency, by selecting menu option 13.

Throughput Scaling via Parallelism

Once the core functionality was realized and verified, the matrix accelerator's throughput was increased by performing MAC operations in parallel. The first iteration featured 2 parallel MAC units, and the second and final iteration featured 4 parallel MAC units. Writing

the HDL to infer parallel processing was trivial; the challenge lied within increasing the BRAM units' data widths in such a way that both hardware and software could access and unpack data from each data port.

Vivado defaults BRAM data widths to 32-bits; with each value represented by a 16-bit net, it was no issue to concatenate 2 16-bit values into a 32-bit word. However, scaling the hardware by a factor of 4 required BRAM units A and B to increase their word size to 64, allowing software to transfer 4 elements per write while hardware slices each word into 4 separate 16-bit nets. Subsequently, a packing helper function was implemented in software to shift each of 4 16-bit integers neatly into a 64-bit integer. Since the hardware performs 4 simultaneous MAC operations per word, the matrix size is limited to the first 4 multiples of 4. Furthermore, matrix C was kept at a 32-bit word size since each 4-MAC operation yields a single 32-bit element for a 4x4 matrix; every 2 4-MAC operation yields a single 32-bit element for an 8x8 matrix, and so on. The performance benefits are discussed in the following section.

The final Vivado block design iteration for the 4-MAC is included below in Fig. 4 for reference.

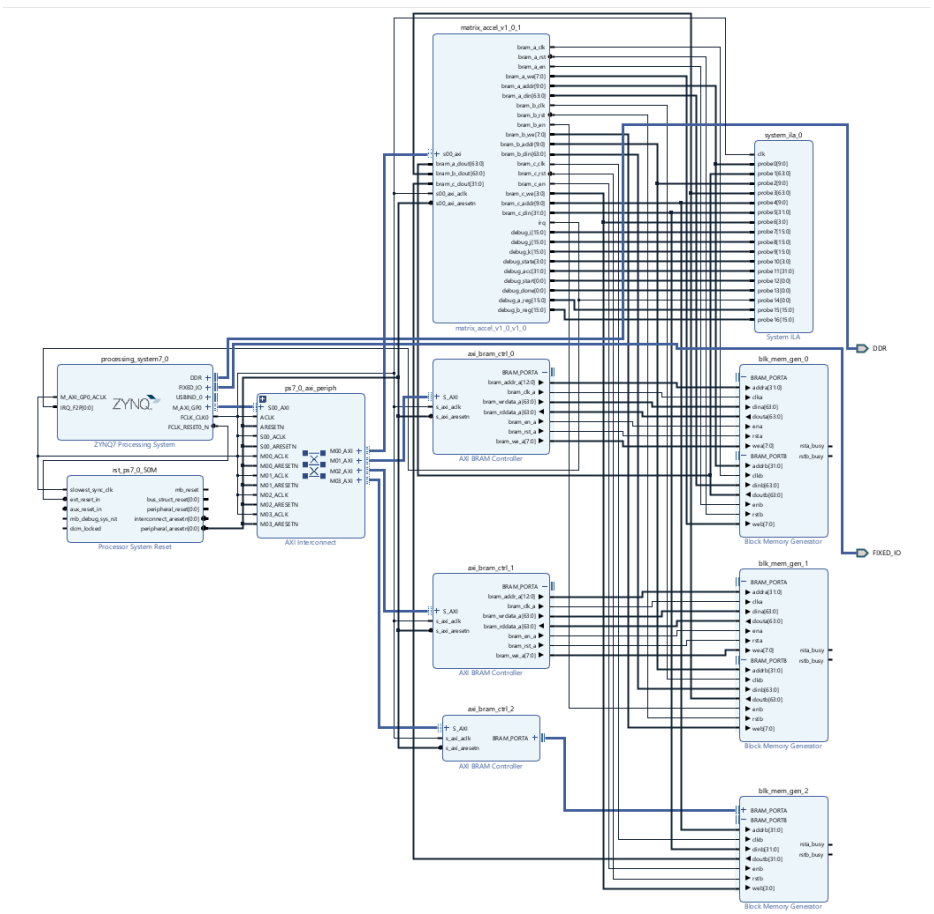

Fig. 4. Vivado block design for the 4-MAC with BRAM A and B featuring a 64-bit data bus.

Analysis and Results

At first, I assumed that the BRAM units were bit-addressed. I used an ILA block to monitor internal PL signals for debugging purposes throughout the design process; Fig. 5 depicts the ILA waveforms corresponding to an erroneous result where only the first row of Matrix C_HW was populated, albeit with incorrect values.

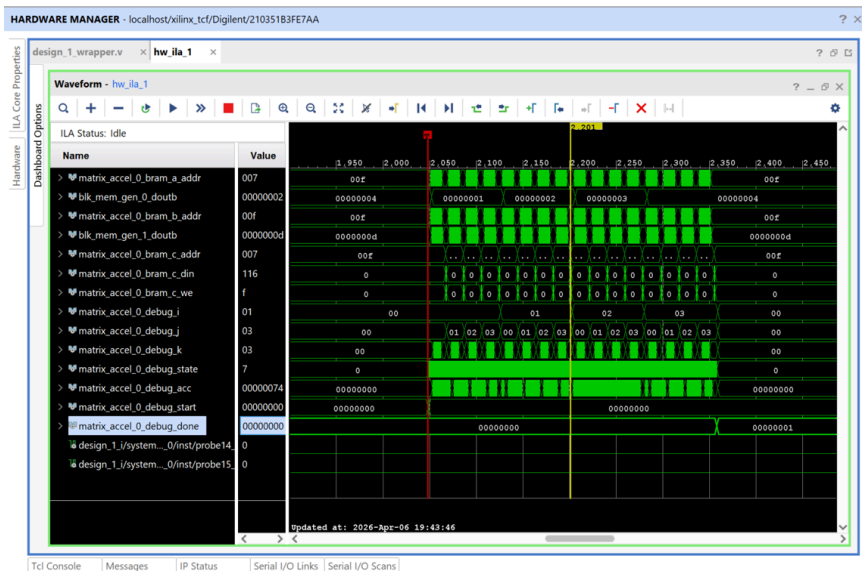

Fig. 5. 1-MAC ILA output waveforms depicting that the output of BMG_0 (BRAM A) remained constant for many calculation cycles while BMG_1 incremented.

It was not until I attempted byte-addressing that caused BMG_0's output to increment correctly and mirror BMG_1's output. Correspondingly, debug_k's ILA waveform now mirrored that of the BMG units' outputs, as depicted in Fig. 6.

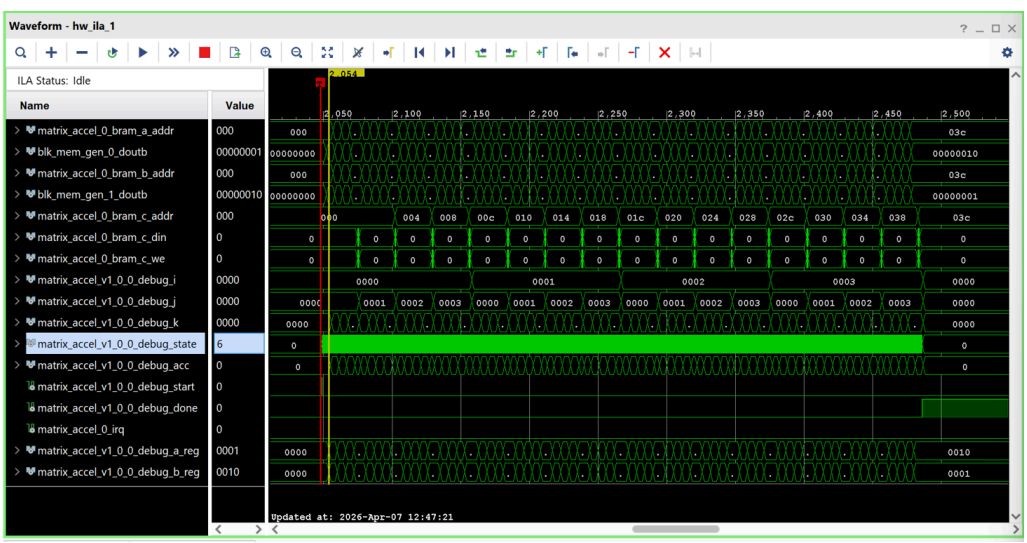

Fig. 6. 1-MAC ILA output waveforms depicting the correct functionality of the matrix acceleration hardware unit.

Fig. 7 depicts the corresponding hardware utilization report for the 1-MAC design. These were documented to provide insight regarding the hardware cost as parallelism is introduced.

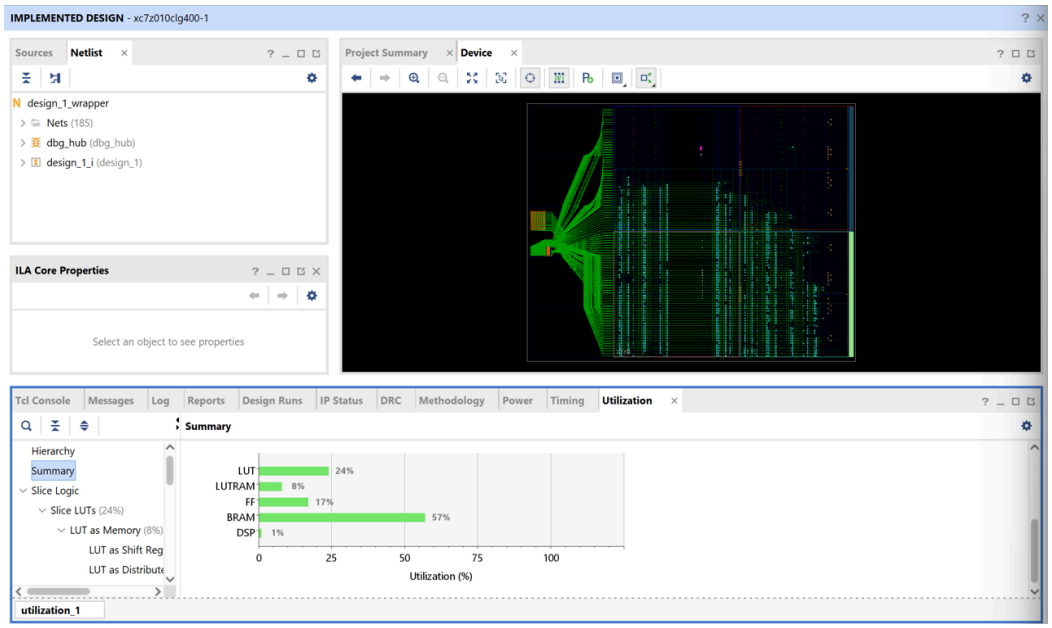

Fig. 7. Utilization report for the 1-MAC design.

As the 1-MAC was modified to become a 2-MAC, we observe that each full matrix multiplication operation takes considerably less clock cycles.

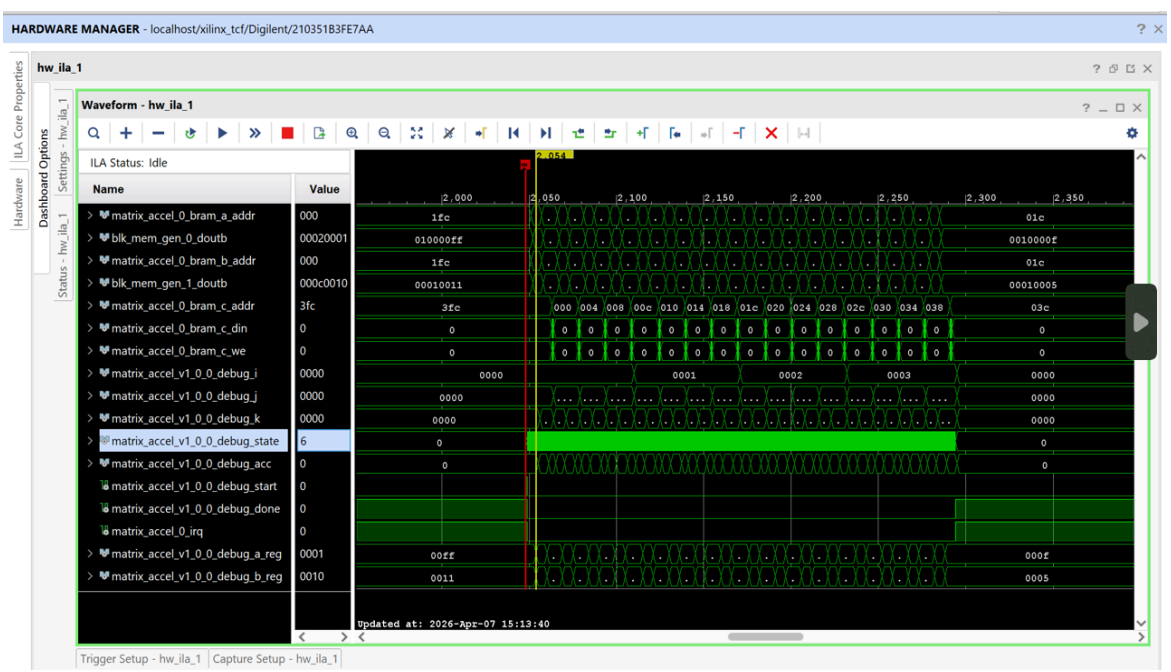

Fig. 8. 2-MAC ILA output waveforms depicting that the full 4x4 operation takes just over 200 clock cycles less when compared to the 1-MAC.

The corresponding hardware utilization report is additionally included below.

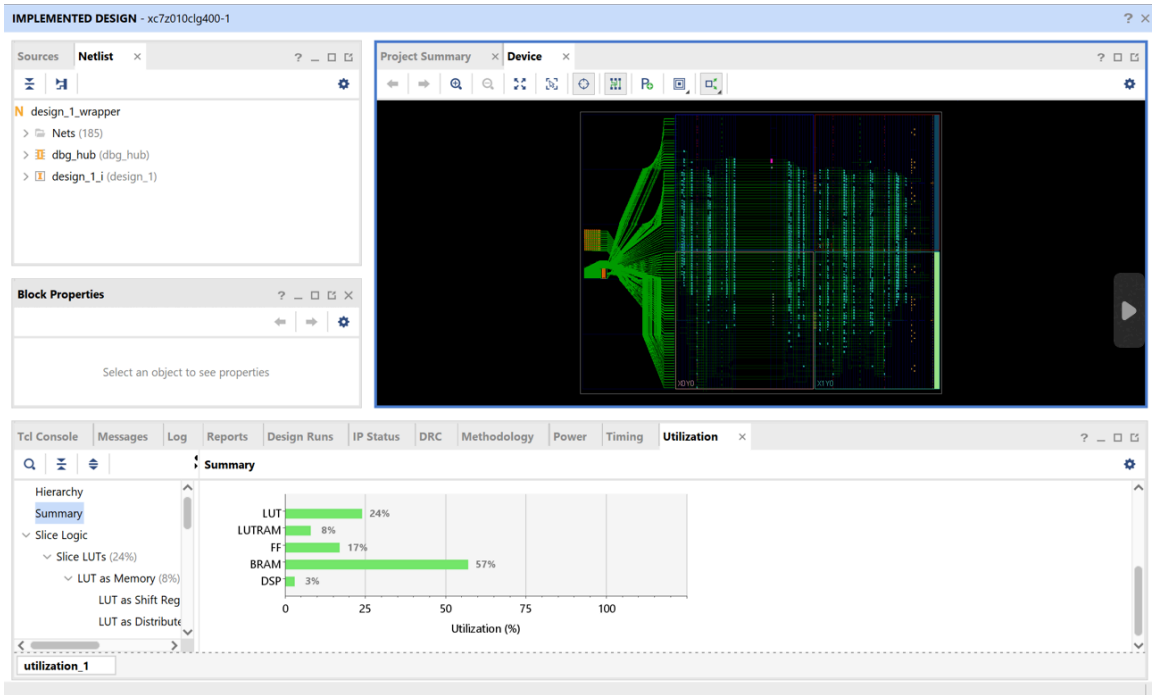

Fig. 9. Utilization report for the 2-MAC design. We observe a 2% increase in DSP resources used.

Finally, with the introduction of the 4-MAC design, we observe in Fig. 10 that each 4x4 operation takes around 100 less clock cycles than an equivalent operation for the 2-MAC.

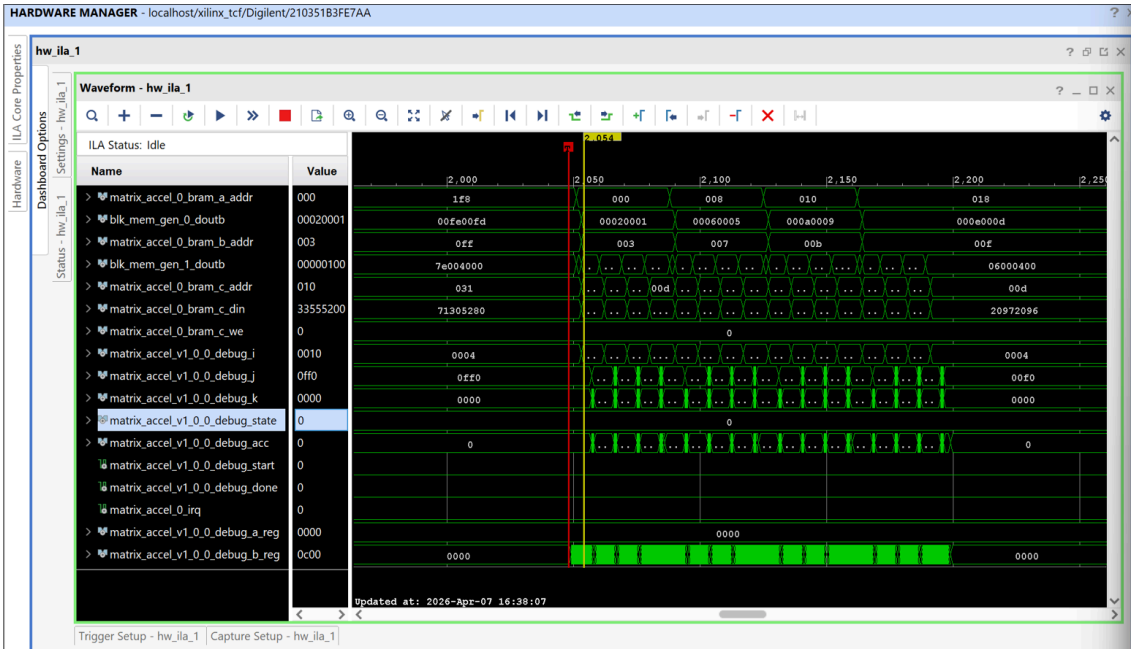

Fig. 10. 4-MAC ILA output waveform depicting that a 4x4 matrix operation is marginally faster using the 4-MAC hardware when compared to the 2-MAC design.

The corresponding 4-MAC hardware utilization report is included below.

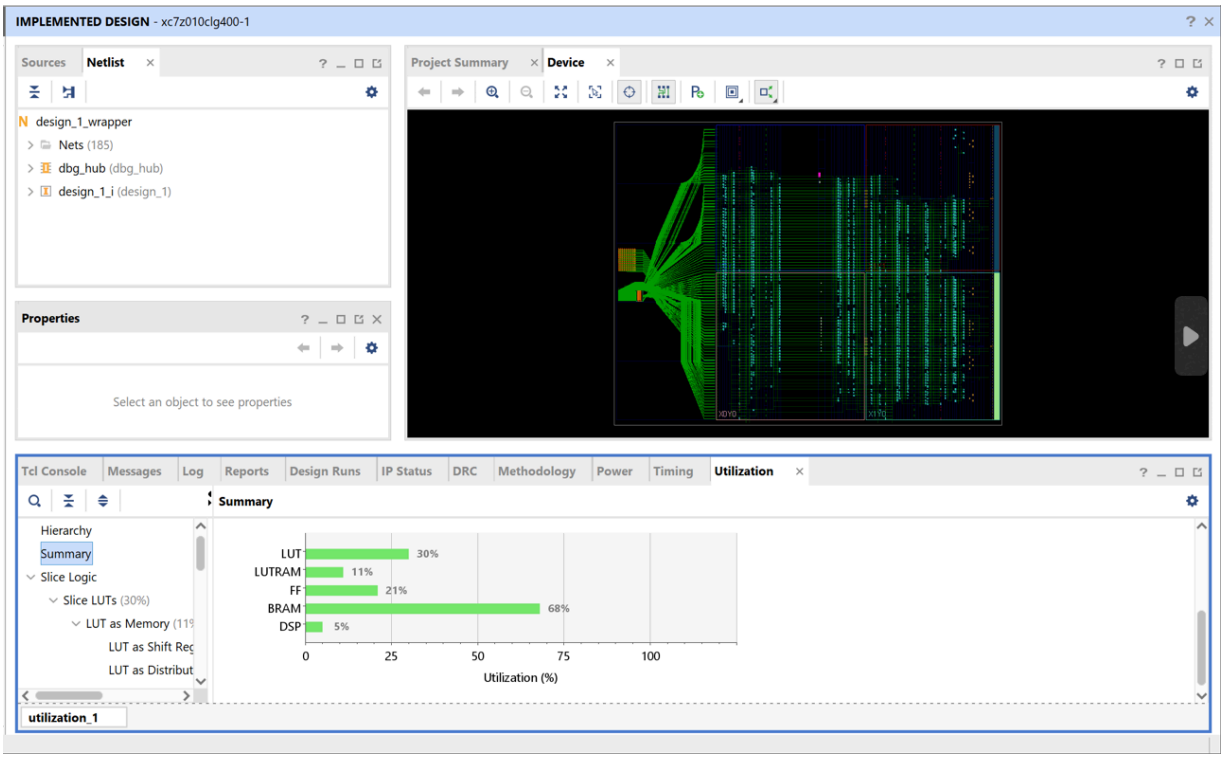

Fig. 11. Utilization report for the 4-MAC design. We observe a 6% increase in LUT resources, a 3% increase in LUTRAM resources, a 4% increase in flip flop resources, an 11% increase in BRAM resources, and a 2% increase in DSP resources used.

Using the menu user interface to test the performance of each iteration of the matrix acceleration hardware, we can test how increasing the matrix size and the parallelism employed affects the number of clock cycles necessary to fully complete each operation.

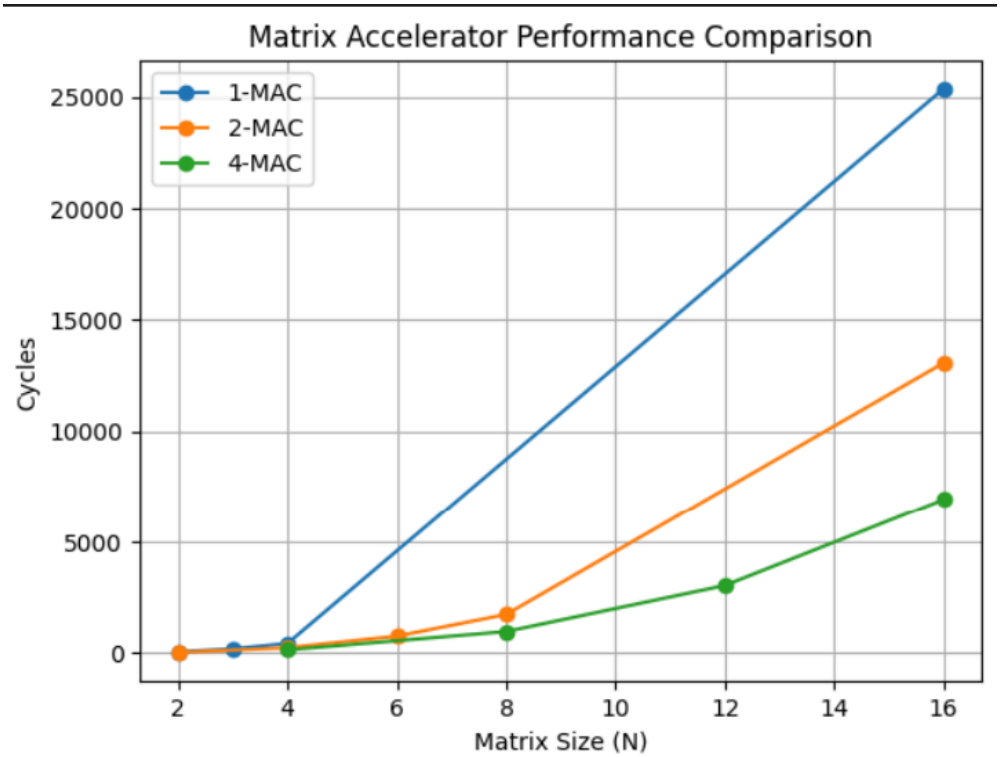

Fig. 12. Line plot comparing the number of clock cycles per operation to the size of the matrix operation for all 3 MAC iterations. We observe that there is an exponential relationship between the number of clock cycles required and the size of the matrix operation. Additionally, we observe a large decrease in the total number of clock cycles required per operation between the 1-MAC and 2-MAC, and a lesser decrease between the 2-MAC and the 4-MAC.

While Fig. 12 is useful for determining the relative performance of each hardware iteration, Fig. 13 compares the hardware's performance to the software's fixed performance with a CPU clock frequency of 667 MHz. This was accomplished via comparing the microseconds per operation across all software and hardware iterations as opposed to clock cycles, which varies from hardware to software.

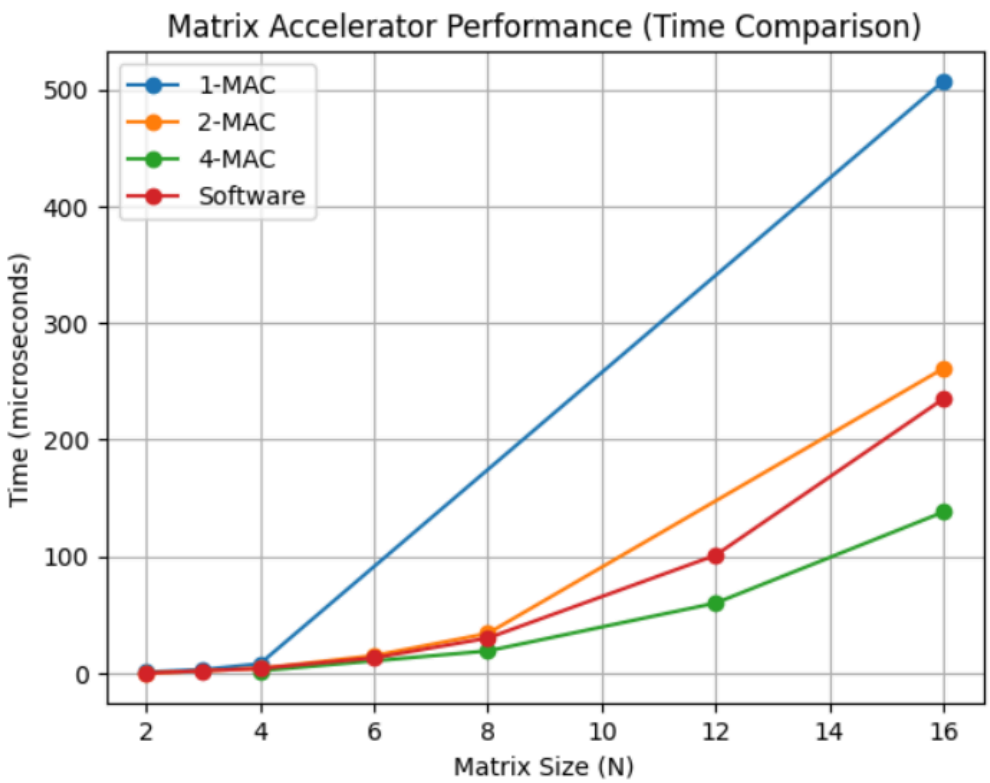

Fig. 13. Line plot comparing the number of microseconds per operation to the size of the matrix operation for all 3 MAC iterations and software. We observe that the software's performance is between that of the 2-MAC and 4-MAC, but closer to the 2-MAC's performance.

When I initially saw the performance difference between software and the 1-MAC, I knew I needed to leverage the PL's hardware resources more optimally. Observing that software running at a 667 MHz clock frequency is outperformed by the 4-MAC at a mere 50 MHz highlights the sheer power of parallelism when incorporated into FPGA design.

Conclusion

This project explored many concepts related to digital logic design including AXI4 handshaking interfaces, PS-PL communication techniques, PL-based parallelism scaling, BRAM-based memory architectures, finite state machine control design, AXI-Lite slave register designation, and hardware-software co-design for performance optimization. The skills used throughout this design process can be used in a variety of other digital logic applications where custom memory interfaces, hardware acceleration, and high-throughput parallel data processing are required.

References

[1] AMD Xilinx, *AXI Reference Guide*, UG1037.

[2] AMD Xilinx, *Zynq-7000 SoC Technical Reference Manual*, UG585.

[3] AMD Xilinx, *Block Memory Generator v8.4 Product Guide*, PG058.

[4] AMD Xilinx, *AXI BRAM Controller v4 Product Guide*, PG078.

[5] AMD Xilinx, *Vivado Design Suite User Guide: Designing with IP*, UG896.